

MPCA

Microprocessor & Computer Architecture

Short Notes — All Units | ESA Preparation

UNIT 1: Architecture & ARM Assembly

1.1 RISC vs CISC

Feature	RISC	CISC
Instruction set	Small, simple, uniform	Large, complex, variable
Instruction size	Fixed (32-bit in ARM)	Variable length
Addressing modes	Few (load/store only)	Many complex modes
Execution	1 cycle per instruction (ideal)	Multi-cycle for complex ops
Registers	Many general-purpose (16 in ARM)	Fewer, special-purpose
Compiler	Complex compiler	Simple compiler
Examples	ARM, MIPS, RISC-V	x86, x86-64

RISC Features ARM KEEPS

- Load/Store architecture — only LDR/STR access memory
- Fixed 32-bit instruction width
- Large register file (R0–R15)
- 3-address instructions: ADD Rd, Rn, Op2

RISC Features ARM REJECTS

- Pure register-to-register — ARM allows LDR Rd,[Rn,#offset] (base+offset)
- Single addressing mode — ARM has pre/post-index, shifted register modes
- No conditional execution — ARM adds condition codes to ALL instructions
- No variable-length instructions — ARM adds Thumb (16-bit) mode

1.2 ARM Register File

Register	Name	Role
R0–R3	a1–a4	Argument/result registers
R4–R11	v1–v8	Variable (callee-saved)
R12	IP	Intra-procedure scratch
R13	SP	Stack Pointer
R14	LR	Link Register (return address)
R15	PC	Program Counter
CPSR	—	Condition flags: N, Z, C, V

1.3 Data Processing Instructions

Syntax: <MNEMONIC>{cond}{S} Rd, Rn, Operand2

Instruction	Operation	Example
ADD	$Rd = Rn + Op2$	ADD R0, R1, R2
SUB	$Rd = Rn - Op2$	SUB R0, R1, #5
RSB	$Rd = Op2 - Rn$ (reverse subtract)	RSB R0, R1, #0 $\rightarrow R0 = -R1$
AND	$Rd = Rn \text{ AND } Op2$ (bit-clear)	AND R0, R1, R2
ORR	$Rd = Rn \text{ OR } Op2$ (bit-set)	ORR R0, R1, #0xFF
EOR	$Rd = Rn \text{ XOR } Op2$ (bit-flip)	EOR R0, R1, R2
BIC	$Rd = Rn \text{ AND } (\text{NOT } Op2)$ (bit clear)	BIC R0, R1, #0x0F clears low nibble
MOV	$Rd = Op2$	MOV R0, #10
MVN	$Rd = \text{NOT } Op2$	MVN R0, #0 $\rightarrow R0 = 0xFFFFFFFF$
CMP	Sets flags from $Rn - Op2$, no dest	CMP R1, #0
TST	Sets flags from $Rn \text{ AND } Op2$	TST R0, #1 (test bit 0)
MUL	$Rd = Rm * Rs$ (32-bit result)	MUL R0, R1, R2

1.4 C to ARM Compilation

Example: $F = (G + H) - (I + J)$

Assume $G=R0, H=R1, I=R2, J=R3, F=R4$ (or use memory):

```

ADD R5, R0, R1      ; R5 = G + H
ADD R6, R2, R3      ; R6 = I + J
SUB R4, R5, R6      ; F = (G+H) - (I+J)

```

If-else pattern: $\text{if}(a > b) x = a - b; \text{else } x = b - a;$

```

CMP R0, R1          ; compare a, b
SUBGT R2, R0, R1    ; if a>b: x = a-b
SUBLE R2, R1, R0    ; if a<=b: x = b-a

```

Key insight: the condition suffix (GT, LE, EQ, NE, GE, LT) replaces branching entirely.

1.5 Conditional Execution

Converting branch-based code to conditional instructions:

Original (with branches):

```

CMP R3, R4
BNE ELSE
ADD R0, R1, R2
B EXIT
ELSE: SUB R0, R1, R2
EXIT: ...

```

After conditional execution:

```

CMP R3, R4

```

```
ADDEQ R0, R1, R2    ; execute if R3 == R4
SUBNE R0, R1, R2    ; execute if R3 != R4
```

Suffix	Meaning	Flags
EQ	Equal	Z=1
NE	Not Equal	Z=0
GT	Greater Than (signed)	Z=0, N=V
LT	Less Than (signed)	N≠V
GE	Greater or Equal (signed)	N=V
LE	Less or Equal (signed)	Z=1 or N≠V
CS/HS	Carry Set / Higher or Same (unsigned)	C=1
CC/LO	Carry Clear / Lower (unsigned)	C=0

1.6 Load / Store Instructions

Instruction	Operation
LDR Rd, [Rn]	Load 32-bit word from address in Rn
LDR Rd, [Rn, #off]	Load from Rn+offset (pre-index, no writeback)
LDR Rd, [Rn, #off]!	Pre-index WITH writeback: Rn = Rn+off first
LDR Rd, [Rn], #off	Post-index: load from Rn, then Rn = Rn+off
LDRB / LDRH	Load byte / halfword (zero-extended)
STR Rd, [Rn, #off]	Store 32-bit word
STRB / STRH	Store byte / halfword
LDMFD R13!, {R4-R7}	Load multiple (Full Descending) — POP
STMFD R13!, {R4-R7}	Store multiple (Full Descending) — PUSH

Stack Conventions

- Full Descending (FD): SP points to last pushed item, grows downward
- PUSH = STMFD R13!, {regs}
- POP = LDMFD R13!, {regs}

1.7 Memory Organisation & Endianness

Data Directives

Directive	Size	Example
.byte	1 byte each	.byte 20, 50, 38, 45
.halfword	2 bytes each	.halfword 4567 → stored as 0x11D7
.word	4 bytes each	.word 78650A09

Little-Endian Layout (ARM default)

LSB (Least Significant Byte) stored at LOWEST address.

Example: .word 0x78650A09 starting at address 0x00306090:

Address	Contents
0x00306090	0x09 (LSB)
0x00306091	0x0A
0x00306092	0x65
0x00306093	0x78 (MSB)

1.8 Subroutines & Parameter Passing

Call mechanism

```
BL SUBROUTINE    ; Branch & Link: saves PC+4 into LR (R14)
MOV PC, LR       ; Return: restore PC from LR
```

Parameter passing methods

- Via registers (R0–R3 for up to 4 args) — fastest
- Via stack (STMFD before call, LDMFD inside subroutine) — for more args
- Via memory (pass address in register)

Stack-based example

```
MOV R1, #52
MOV R2, #25
STMFD R13!, {R1,R2}    ; push args
BL LINK
LINK: LDMFD R13!, {R6,R5} ; pop into R6=52, R5=25
      ADD R0, R6, R5
      MOV PC, LR
```

1.9 ARM Instruction Encoding

32-bit format: [31:28]=cond [27:26]=00 [25]=I [24:21]=opcode [20]=S [19:16]=Rn [15:12]=Rd [11:0]=operand2

To decode hex: convert to binary, extract fields, look up cond/opcode tables.

Example: E3B00008 → 1110 0011 1011 0000 0000 0000 0000 1000

- cond=1110 (AL, always) [27:26]=00 I=1 (immediate)
- opcode=1101 (MOV) S=1 (set flags) Rn=0000 Rd=0000 imm=8
- Instruction: MOVS R0, #8

UNIT 2: Pipelining

2.1 Pipeline Basics

- Pipelining overlaps execution of multiple instructions
- Each stage takes 1 clock cycle; clock period = max stage delay + register overhead

5-Stage ARM Pipeline

Stage	Name	What happens
IF	Instruction Fetch	Fetch instruction from memory using PC
ID	Instruction Decode	Decode opcode, read registers, extend immediates
EX	Execute	ALU operation or address calculation
MEM	Memory Access	Load/store data from/to memory
WB	Write Back	Write result to destination register

Performance Formulas

Clock cycle time (pipelined) = max(stage delays) + register overhead

Throughput = 1 instruction / clock cycle (ideal steady-state)

Latency (one instruction) = $N \times \text{clock cycle}$ (N = number of stages)

Speedup $\approx N$ stages (ideal, no hazards)

CPI (with hazards) = 1 + stall cycles per instruction

Total clock cycles = ($N_{\text{instructions}} + N_{\text{stages}} - 1$) + stall cycles

2.2 Pipeline Hazards

1. Structural Hazard

Two instructions need the same hardware resource at the same time.

- Example: unified cache — IF and MEM both need memory in same cycle
- Solution: separate I-cache and D-cache, or stall
- CPI penalty: if 30% instructions are loads, stall = 0.3 cycles extra

2. Data Hazard (most common exam topic)

An instruction needs a result not yet written by an earlier instruction.

Type	Description	Example
RAW (Read After Write)	Most common. Instruction 2 reads before Instruction 1 writes.	ADD R1,R2,R3 SUB R4,R1,R5
WAW (Write After Write)	Two writes to same reg out of order	ADD R1,R2,R3 SUB R1,R4,R5
WAR (Write After Read)	Write before upstream read completes	SUB R4,R1,R5 ADD R1,R2,R3

Resolving Data Hazards

- Forwarding (bypassing): route result directly from EX/MEM output to EX input. Resolves RAW in most cases without stalls.
- Stalling (pipeline bubble): if result not yet available even with forwarding (e.g. load-use hazard). 1 stall cycle inserted.

Load-use hazard example (requires 1 stall even with forwarding):

```
LDR R1, [R2]      ; result available after MEM stage
ADD R3, R1, R4     ; needs R1 in EX — one cycle too early
NOP               ; compiler inserts stall / reorder
```

3. Control Hazard

Branch instruction causes uncertainty about which instruction to fetch next.

- Solution 1: Flush pipeline (simple, costly — wastes branch penalty cycles)
- Solution 2: Branch prediction
- Solution 3: Delayed branching (ARM uses this — instruction after branch always executes)

2.3 Branch Prediction

1-bit Predictor

- State: T or NT. Predicts same as last outcome.
- Mispredicts twice in a loop on entry and exit.

2-bit Predictor (Saturating Counter)

States: 11 (Strongly T) → 10 (Weakly T) → 01 (Weakly NT) → 00 (Strongly NT)

States 11 and 10 predict TAKEN; states 01 and 00 predict NOT TAKEN.

Transitions: T moves toward 11 end; NT moves toward 00 end.

Tracing procedure (exam technique):

- Write current state for each branch in the sequence
- Predict based on state (11,10 = T; 01,00 = NT)
- Compare prediction with actual: if wrong = misprediction
- Update state based on actual outcome

Choose starting state that minimises mispredictions: for a sequence heavy on T, start at 11.

2.4 Pipeline Performance Calculations

Speedup of pipelining

$\text{Speedup} = \text{Time_nonpipelined} / \text{Time_pipelined}$

Non-pipelined time for N instructions = $N \times (\text{sum of all stage delays})$

Pipelined time = $(N + \text{stages} - 1) \times \text{clock_cycle}$

Structural hazard CPI example

Given: ideal CPI=1, structural hazard from unified cache, 30% data accesses:

$$\text{CPI}_{\text{actual}} = 1 + 0.30 \times 1 = 1.30$$

$$\text{Speedup} = (5 \times \text{old_clock}) / (1.30 \times \text{new_clock})$$

Mixed stage delay example

For 100 instructions: 40 take 3 cycles in EX, 35 take 2 cycles, 25 take 1 cycle:

$$\text{Base cycles} = (\text{IF} + \text{ID} + \text{WB} + \text{WR}) \times 100 = 4 \times 100 = 400$$

$$\text{EX cycles} = 40 \times 3 + 35 \times 2 + 25 \times 1 = 120 + 70 + 25 = 215$$

$$\text{Total} = 400 + 215 = 615 \text{ clock cycles}$$

2.5 Five-Stage Pipeline Timing

$$\text{Cycle time} = \max(\text{stage_delay}) + \text{register_overhead}$$

Example: Fetch=300ps, Decode=400ps, Execute=350ps, Memory=550ps, WB=100ps, reg=20ps

- Cycle time = $\max(300, 400, 350, 550, 100) + 20 = 570\text{ps}$
- Latency = $5 \times 570 = 2850\text{ps}$
- Throughput = $1/570\text{ps} \approx 1.75 \text{ GIPS}$

UNIT 3: Cache Memory

3.1 Locality of Reference

Type	Definition	Example
Temporal	A location accessed recently is likely to be accessed again soon	Loop variable i accessed every iteration
Spatial	Locations near a recently accessed location are likely to be accessed	Array elements processed sequentially

Significance: locality enables small, fast cache to deliver near-memory performance.

3.2 Cache Terminology

Term	Definition
Hit	Data found in cache
Miss	Data not in cache — must fetch from main memory
Hit time	Time to access data when found in cache
Miss penalty	Extra time to fetch block from memory on a miss
Miss rate	Fraction of accesses that miss = misses / total accesses
Hit rate	$1 - \text{miss rate}$
AMAT	$\text{Average Memory Access Time} = \text{hit_time} + \text{miss_rate} \times \text{miss_penalty}$
Bandwidth	Rate of data transfer between cache and memory
Read miss	Cache miss on a load instruction
Write miss	Cache miss on a store instruction

3.3 Cache Organisation

Direct Mapped

- Each memory block maps to exactly ONE cache line
- $\text{Line index} = (\text{block_number}) \bmod (\text{number_of_lines})$
- Simple, fast, but suffers conflict misses

Fully Associative

- A block can go into ANY cache line
- No conflict misses, but expensive hardware comparators

N-way Set Associative

- Cache divided into sets; each set has N lines
- Block maps to one set, can occupy any of N lines in that set
- Balance between direct-mapped and fully associative

3.4 Address Bit Splitting (KEY exam topic)

Given: address size A bits, block size B bytes, number of lines L, N-way sets

Field	Bits	Formula
Byte offset	b	$b = \log_2(\text{block_size_in_bytes})$
Line / Index	l	$l = \log_2(\text{number_of_lines} / N_ways) = \log_2(\text{number_of_sets})$
Tag	t	$t = A - l - b$

Worked example

128 MB cache, 32 lines per set, 64 bytes per line, 32-bit address:

- Byte offset: $\log_2(64) = 6$ bits
- Total lines = $128\text{MB} / 64\text{B} = 2\text{M}$ lines
- Sets = $2\text{M} / 32 = 64\text{K}$ sets $\rightarrow$ Index = $\log_2(64\text{K}) = 16$ bits
- Tag = $32 - 16 - 6 = 10$ bits
- When a block is loaded, 64 bytes are copied into cache

Another worked example (set associative)

16MB byte-addressable memory, 32KB 4-way SA cache, block size 512 bytes:

- Address bits = $\log_2(16\text{M}) = 24$ bits
- Byte offset = $\log_2(512) = 9$ bits
- Total blocks in cache = $32\text{KB} / 512\text{B} = 64$ blocks
- Number of sets = $64 / 4 = 16 \rightarrow$ Index = $\log_2(16) = 4$ bits
- Tag = $24 - 4 - 9 = 11$ bits
- Block number of address A = $A / 512$ (integer division)
- Set = $\text{block_number} \bmod 16$

3.5 Cache Replacement Policies

LRU (Least Recently Used)

Replace the block that was accessed least recently. Best performance, more hardware.

FIFO (First In First Out)

Replace the block that has been in cache the longest. Simpler.

Example: 2-way SA, 8 blocks (4 sets), FIFO

Memory blocks: 4, 3, 25, 8, 19, 6, 25, 8 ...

Set = $\text{block_number} \bmod 4$. Block 4 $\rightarrow$ set 0; block 3 $\rightarrow$ set 3; block 25 $\rightarrow$ set 1 ...

Track entries per set in FIFO order; on eviction, remove oldest entry.

3.6 Cache Performance Calculations

AMAT formula

$$\text{AMAT} = \text{hit_time} + \text{miss_rate} \times \text{miss_penalty}$$

CPI with cache misses

$$\text{CPI_actual} = \text{CPI_ideal} + \text{mem_stall_cycles_per_instruction}$$

$$\text{Stall cycles} = \text{instruction_miss_rate} \times \text{miss_penalty} + \text{data_miss_rate} \times \text{data_fraction} \times \text{miss_penalty}$$

Split vs Unified cache example

Split: I-cache miss 2%, D-cache miss 10%, data refs 35%, miss penalty 9 cc:

$$\text{Stalls} = 0.02 \times 9 + 0.35 \times 0.10 \times 9 = 0.18 + 0.315 = 0.495 \text{ stall cycles/instruction}$$

Unified: miss 3%, miss penalty 9 cc (but structural hazard adds 0.35 stall):

$$\text{Stalls} = (1 + 0.35) \times 0.03 \times 9 = 0.3645 \text{ stall cycles/instruction}$$

Unified is better here (lower stalls).

Memory bandwidth example

$$\text{Overhead} = 200\text{cc}, 64 \text{ bytes every } 8\text{cc} \rightarrow \text{block of } 1024\text{B} = 200 + (1024/64) \times 8 = 200 + 128 = 328\text{cc}$$

$$\text{AMAT} = 1 + 0.0185 \times 328 = 1 + 6.068 = 7.07 \text{ cc}$$

3.7 Six Basic Cache Optimisations

Technique	Goal	Tradeoff
1. Larger block size	Reduce miss rate (spatial locality)	Increases miss penalty; may cause pollution
2. Higher associativity	Reduce miss rate (conflict misses)	Increases hit time
3. Victim cache	Reduce miss rate (small fully-assoc buffer)	Small hardware cost
4. Prefetching	Reduce miss penalty (fetch before needed)	May pollute cache
5. Write buffer	Reduce miss penalty (absorbs write stalls)	Buffer must not be full
6. Critical word first	Reduce miss penalty (send needed word first)	Complexity

Write policies

Policy	On write hit	On write miss
Write-through	Write to both cache and memory simultaneously	Write allocate OR no-write allocate
Write-back	Write only to cache; memory updated on eviction (dirty bit)	Fetch block to cache, then write (allocate)

UNIT 4: Parallel Computing, I/O & Architecture

4.1 Interrupts

What is an interrupt?

An asynchronous signal from hardware or software requesting CPU attention. CPU suspends current task, saves state, services interrupt, then resumes.

Types of interrupts in ARM

Type	Description
FIQ (Fast IRQ)	High-priority. Dedicated banked registers (R8–R14_fiq). Minimal context save. Used for time-critical tasks.
IRQ (Interrupt Request)	Normal priority. General-purpose interrupts.
SWI (Software Interrupt)	OS system calls. Triggered by SWI instruction.
Abort	Memory access fault (prefetch or data abort).
Undefined	Unrecognised instruction.

Enable / Disable FIQ in ARM

```
MRS R0, CPSR          ; read current CPSR
BIC R0, R0, #0x40      ; clear bit 6 (FIQ mask) — ENABLE FIQ
MSR CPSR_c, R0

MRS R0, CPSR          ; read current CPSR
ORR R0, R0, #0x40      ; set bit 6 (FIQ mask) — DISABLE FIQ
MSR CPSR_c, R0
```

IRQ is bit 7 (0x80). Both can be masked together with 0xC0.

Non-nested interrupt handler flowchart

- 1. CPU detects interrupt signal
- 2. Complete current instruction
- 3. Save PC and CPSR onto stack (or into banked regs)
- 4. Set PC to interrupt vector address
- 5. Execute ISR (Interrupt Service Routine)
- 6. Restore PC and CPSR — return to interrupted program

4.2 Multiple Interrupt Handling

Daisy Chain

- Devices connected in series on a single interrupt line
- CPU sends ACK signal; propagates through chain
- First device with a pending interrupt captures ACK, sends its ID
- Priority determined by position in chain (closer = higher priority)
- Drawback: low-priority device can block high-priority if it captures ACK first

Programmable Interrupt Controller (PIC)

- Dedicated chip (e.g., Intel 8259A) between devices and CPU
- Has interrupt mask register, priority resolver, interrupt vector generation
- CPU programs PIC via I/O ports to set priorities and masks
- PIC selects highest-priority pending interrupt, sends IRQ to CPU
- CPU ACKs; PIC puts vector on data bus
- Advantage: flexible, software-programmable priorities

Interrupt Vector Table

A fixed memory region containing addresses (vectors) of ISRs. Each interrupt type has a fixed entry. On interrupt, CPU reads the vector and jumps to ISR.

ARM vector table at address 0x00000000: Reset(0x00), Undef(0x04), SWI(0x08), Prefetch(0x0C), Data(0x10), IRQ(0x18), FIQ(0x1C)

4.3 DMA (Direct Memory Access)

Why DMA?

CPU-driven I/O wastes cycles busy-waiting (polling) or handling frequent interrupts for large data transfers. DMA controller handles bulk transfers autonomously.

DMA Controller components

- Address register: source/destination memory addresses
- Word count register: number of bytes to transfer
- Control register: direction (read/write), mode

Cycle Stealing

- DMA temporarily takes control of the memory bus for one cycle
- CPU is stalled for that cycle (bus denied)
- DMA transfers one word, releases bus
- CPU resumes; DMA repeats until transfer complete
- Advantage: minimal CPU involvement; CPU not completely stopped

DMA vs Interrupt-driven I/O vs Polling

Method	CPU involvement	Best for
Polling	Constant — wastes cycles	Simple, infrequent I/O
Interrupt-driven	Interrupted per word/byte	Low-volume I/O
DMA	Only setup + completion interrupt	High-volume bulk transfers

4.4 Instruction Level Parallelism (ILP)

Definition

ILP is the average number of instructions that can be executed simultaneously per clock cycle. It exploits independent instructions within a single instruction stream.

ILP Factor calculation

Count independent instruction groups (instructions with no dependency on each other in the same cycle):

Example: 1. $e=a+b$ 2. $f=c+d$ 3. $g=e*f$

- Cycle 1: instructions 1 & 2 are independent → execute in parallel
- Cycle 2: instruction 3 depends on 1 & 2 → must wait
- Total cycles = 2, Total instructions = 3, $ILP = 3/2 = 1.5$

Scalar vs Superscalar

Feature	Scalar	Superscalar
Instructions/cycle	1	2 or more (multiple pipelines)
Hardware	Single pipeline	Multiple parallel pipelines
ILP exploitation	None within a cycle	Automatic by hardware
CPI	≥ 1	Can be < 1

4.5 Parallel Computing

Flynn's Taxonomy

Class	Instructions	Data	Example
SISD	Single	Single	Classical uniprocessor
SIMD	Single	Multiple	GPU, vector processors, SSE/AVX
MISD	Multiple	Single	Fault-tolerant pipelines (rare)
MIMD	Multiple	Multiple	Multi-core CPUs, clusters

Memory Architectures

- Shared Memory (UMA — Uniform Memory Access): all processors share one address space. Easy to program, bandwidth bottleneck.
- NUMA (Non-Uniform Memory Access): shared address space but local memory is faster. Modern multi-socket servers.
- Distributed Memory: each processor has private memory; communicate via message passing (MPI). Scales to thousands of nodes.
- Multicore: multiple cores on one die sharing L2/L3 cache. Low-latency inter-core communication.

4.6 Amdahl's Law

Speedup is limited by the sequential (serial) portion of a program.

Formula: $\text{Speedup} = 1 / (S + (1-S)/P)$

Where S = serial fraction, P = number of processors

Example: 20% serial code, 32 processors:

$$\text{Speedup} = 1 / (0.20 + 0.80/32) = 1 / (0.20 + 0.025) = 1 / 0.225 = 4.44\times$$

Key insight: even with infinite processors, max speedup = $1/S$

With $S=0.20$: max speedup = $1/0.20 = 5\times$ (regardless of P)

4.7 Gustafson's Law (Scaled Speedup)

Amdahl's assumes fixed problem size. Gustafson says: larger problems are more parallelisable.

Formula: $\text{Scaled_Speedup} = P + (1-P) \times s$

Where s = serial fraction (as fraction of PARALLEL execution time), P = processors

Example: 95% parallelisable ($s=0.05$), $P=32$:

$$\text{Scaled Speedup} = 32 + (1-32) \times 0.05 = 32 - 1.55 = 30.45\times$$

Law	Assumes	When to use
Amdahl's	Fixed problem size; serial fraction fixed	Strong scaling — same workload, more processors
Gustafson's	Problem size scales with processors	Weak scaling — bigger workload with more processors

4.8 Multithreading

Type	Description
Fine-grained (interleaved)	Switches thread every clock cycle. Hides pipeline latency. Requires many hardware thread contexts.
Coarse-grained	Switches only on long-latency events (cache miss). Lower overhead but less latency hiding.
Simultaneous (SMT / Hyper-threading)	Multiple threads issue instructions to different functional units in the SAME cycle.

Advantages of multithreading

- Hides memory latency — while one thread waits for memory, another runs
- Better CPU utilisation — fewer idle cycles
- Improved throughput without increasing clock frequency
- Cheaper than multiple physical processors

4.9 Cache Coherence & False Sharing

Cache Coherence

In a multicore system, each core has its own L1 cache. If Core A and Core B both cache the same memory block, and Core A writes to it, Core B's copy becomes stale.

Protocols like MESI (Modified, Exclusive, Shared, Invalid) ensure all caches see a consistent view of memory.

False Sharing

Two processors access DIFFERENT variables that happen to reside in the SAME cache block. When one processor writes, it invalidates the other's copy — even though the other processor's variable wasn't actually changed.

Fix: pad data structures so variables used by different cores are in separate cache lines.

4.10 OpenMP & Parallel Programming

OpenMP — shared memory parallelism (C/C++/Fortran)

```
#pragma omp parallel for
for(int i=0; i<N; i++) { a[i] = b[i] + c[i]; }
```

Key directives

- `#pragma omp parallel` — fork worker threads
- `#pragma omp for` — distribute loop iterations
- `#pragma omp critical` — mutual exclusion section
- `#pragma omp reduction(+:sum)` — combine partial results

GPU Computing (CUDA/OpenCL)

- GPU has thousands of small cores optimised for SIMD
- Kernel: function that runs on GPU across many threads
- Thread hierarchy: thread → warp (32) → block → grid
- Best for data-parallel workloads: matrix multiply, convolution (CNNs)
- CPU handles serial/control-flow parts; GPU handles data-parallel parts

4.11 Quick Formula Reference

Formula	Equation
AMAT	$\text{hit_time} + \text{miss_rate} \times \text{miss_penalty}$
CPI with memory stalls	$\text{CPI_ideal} + \text{miss_rate} \times \text{miss_penalty}$
Pipeline speedup (ideal)	N stages
Amdahl's speedup	$1 / (S + (1-S)/P)$
Gustafson's scaled speedup	$P + (1-P) \times s$
ILP factor	Total instructions / Total cycles with parallelism

Cache index bits	$\log_2(\text{number_of_sets})$
Cache byte offset bits	$\log_2(\text{block_size_in_bytes})$
Cache tag bits	$\text{Address_bits} - \text{index_bits} - \text{offset_bits}$

— *End of Notes* —